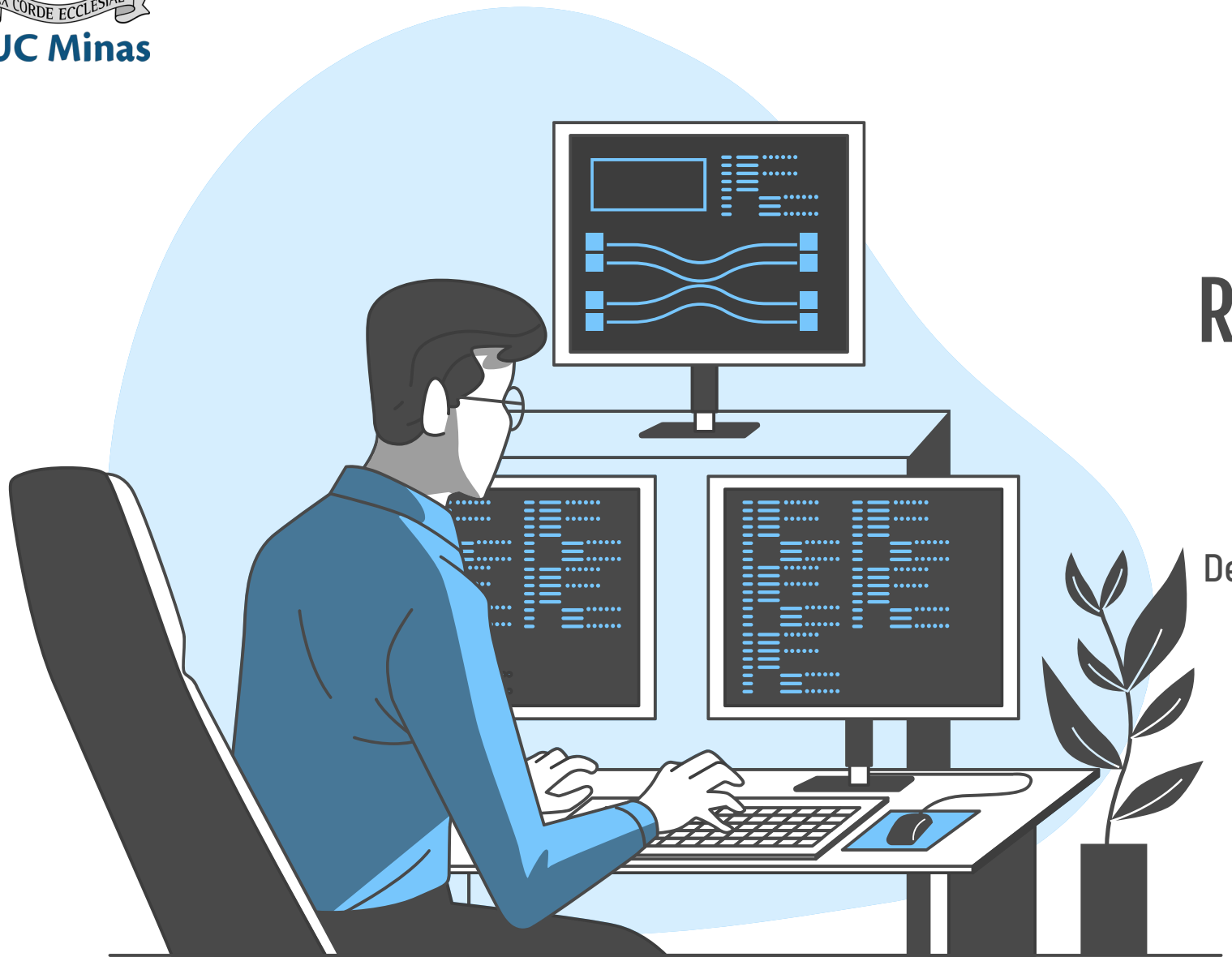
PUC Minas



Aula 08 · Front-end

Rotas com React Router e organização do projeto

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1. Instalar e configurar o **React Router**
2. Criar rotas, `Link`/`NavLink` e página 404
3. Ler parâmetros com `useParams` e navegar com `useNavigate`
4. Criar uma **rota protegida**
5. Organizar `pages/`, `components/`, `services/`, `hooks/`, `config/`, `utils/`

Apresentação da aula

Antes de começar

- Aula 06: services

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 Uma SPA com várias “páginas”

02 Navegação

03 Rota protegida

04 Organização das pastas

05 Hooks personalizados

06 Aprofundando: rotas aninhadas e layouts

01

Uma SPA com várias “páginas”

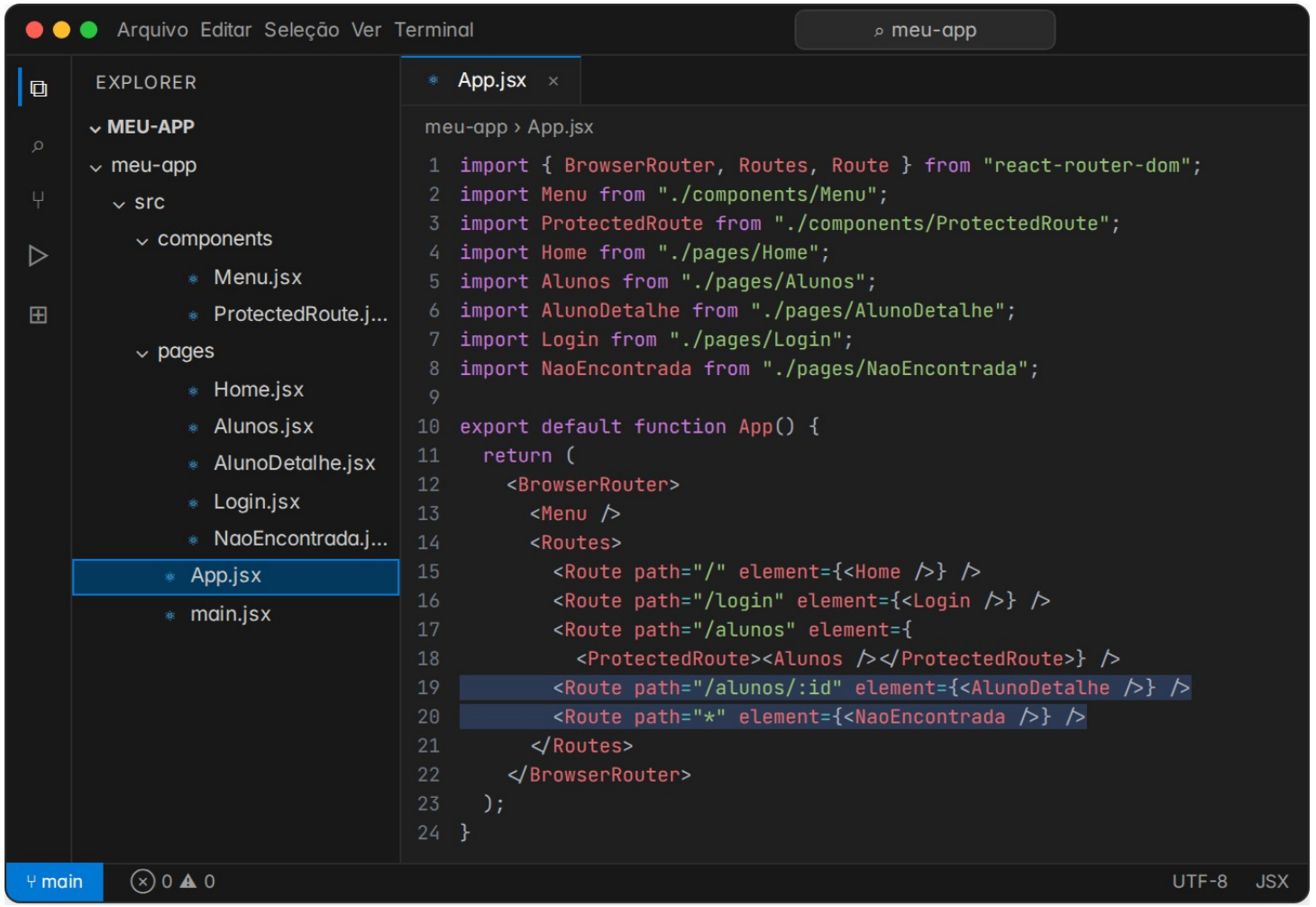
Uma SPA com várias “páginas”

Em uma SPA o servidor entrega sempre o mesmo `index.html`. Quem decide o que mostrar para `/alunos` ou `/sobre` é o JavaScript, lendo a URL do navegador. A biblioteca padrão para isso é o **React Router**.

Instalação

```
npm install react-router-dom
```

Uma SPA com várias “páginas”



Rotas declaradas no `App.jsx`. `*` captura qualquer endereço não cadastrado (página 404).

02

Navegação

Navegação

Nunca use `<a>` para links internos: ele recarrega a página inteira e perde o estado. Use `<Link>` (ou `<NavLink>`, que adiciona a classe `active` no link da página atual).

components/Menu.jsx

```jsx
import { NavLink } from "react-router-dom";

export default function Menu() {
  return (
    <nav>
      <NavLink to="/">Início</NavLink>
      <NavLink to="/alunos">Alunos</NavLink>
      <NavLink to="/login">Entrar</NavLink>
    </nav>
  );
}
```

pages/AlunoDetalhe.jsx

```jsx
import { useParams, useNavigate }
  from "react-router-dom";

export default function AlunoDetalhe() {
  const { id } = useParams();    // /alunos/42
  const navigate = useNavigate();
  // ... buscar aluno pelo id
  return (
    <button onClick={() => navigate(-1)}>
      Voltar
    </button>
  );
}
```

Navegação

Recurso	Uso
``	Link interno
``	Link com estilo de “ativo”
`useParams()`	Lê `:id` da rota
`useNavigate()`	Navega por código: `navigate("/alunos")`, `navigate(-1)`
`useSearchParams()`	Lê/grava a query string `?busca=ana`
``	Redireciona ao renderizar

03

Rota protegida

Rota protegida

Mesmo padrão do projeto `SecureLoginPUC_React`: um componente que confere se há usuário logado e, se não houver, redireciona.

components/ProtectedRoute.jsx

```jsx
import { Navigate, useLocation } from "react-router-dom";
import { useAuth } from "../auth/useAuth";

export default function ProtectedRoute({ children }) {
  const { usuario } = useAuth();          // vem de um Context de autenticação
  const location = useLocation();

  if (!usuario) {
    return <Navigate to="/login" replace state={{ de: location.pathname }} />;
  }
  return children;
}
```

Rota protegida

Atenção

A rota protegida no front **só esconde a tela**: qualquer pessoa consegue ler o JavaScript. A proteção de verdade dos dados é o back-end exigir o token JWT (Back-end, aula 10).

Rota protegida

Erro comum

F5 em `/alunos` dá 404 depois do deploy: o servidor procura um arquivo `/alunos` que não existe. É preciso mandar todas as rotas para o `index.html`: no Vercel com um `vercel.json` (Deploy, aula 05), no nginx com `try_files` (Deploy, aula 04). Em `npm run dev` o Vite já faz isso sozinho.

04

Organização das pastas

Organização das pastas

Os projetos React da disciplina seguem esta convenção (veja `Pokedex` e `SecureLoginPUC_React`):

Pasta	O que guarda	Exemplos dos projetos
`pages/`	Um componente por rota	`Login.jsx`, `Home.jsx`, `Register.jsx`
`components/`	Peças reutilizáveis (cada uma com seu CSS)	`PokemonCard/`, `SearchBar/`, `Message.jsx`
`services/`	Chamadas a APIs	`pokedexService.js`, `userService.js`
`repository/`	Acesso direto a um banco (ex.: Supabase)	`favoriteRepository.js`
`hooks/`	*Hooks* personalizados	`useFavoritos.js`, `useTheme.js`

Organização das pastas (cont.)

Pasta	O que guarda	Exemplos dos projetos
`config/`	Leitura de `import.meta.env`, clientes	`apiConfig.js`, `supabaseClient.js`
`auth/`	Contexto de autenticação	`AuthProvider.jsx`, `authContext.js`
`utils/`	Funções puras de apoio	`masks.js`, `storage.js`
`model/`	Classes/formatos de dados	`Pokemon.js`, `Trainer.js`
`assets/`	Imagens importadas	`pokedex-aberta.webp`

05

Hooks personalizados

Hooks personalizados

Quando a mesma lógica com estado se repete, extraia para uma função `useAlgo`:

hooks/useLocalStorage.js

```
import { useEffect, useState } from "react";

export function useLocalStorage(chave, inicial) {
  const [valor, setValor] = useState(() => {
    const salvo = localStorage.getItem(chave);
    return salvo !== null ? JSON.parse(salvo) : inicial;
  });

  useEffect(() => {
    localStorage.setItem(chave, JSON.stringify(valor));
  }, [chave, valor]);

  return [valor, setValor];
}

// uso: const [tema, setTema] = useLocalStorage("tema", "claro");
```

06

Aprofundando: rotas aninhadas e layouts

Aprofundando: rotas aninhadas e layouts

Rotas aninhadas

```
<Routes>
  <Route element={<Layout />}>                {/* menu + rodapé para todas */}
    <Route index element={<Home />} />        {/* "/" */}
    <Route path="alunos">
      <Route index element={<Alunos />} />                {/* /alunos */}
      <Route path=":id" element={<AlunoDetalhe />} />     {/* /alunos/42 */}
      <Route path="novo" element={<AlunoForm />} />       {/* /alunos/novo */}
    </Route>
  </Route>
  <Route path="*" element={<NaoEncontrada />} />
</Routes>
```

Aprofundando: rotas aninhadas e layouts

components/Layout.jsx

```jsx
import { Outlet, NavLink } from "react-router-dom";

export default function Layout() {
  return (
    <>
      <header>
        <NavLink to="/">Início</NavLink>
        <NavLink to="/alunos">Alunos</NavLink>
      </header>
      <main><Outlet /></main>          {/* a rota filha aparece aqui */}
      <footer>DIAW · PUC Minas</footer>
    </>
  );
}
```

Filtros na URL com useSearchParams

Estado que sobrevive ao F5

```
const [params, setParams] = useSearchParams();
const busca = params.get("busca") ?? "";

<input value={busca}
       onChange={(e) => setParams({ busca: e.target.value })} />
// a URL vira /alunos?busca=ana: dá para compartilhar o link e o F5 mantém o filtro
```

Aprofundando: rotas aninhadas e layouts

Hook / componente	Uso
``	Lugar onde a rota filha é desenhada
`useLocation()`	URL atual (`pathname`, `search`, `state`)
`useNavigate()`	Navegar por código
`useParams()`	Parâmetros `:id`
`useSearchParams()`	Query string
``	Redirecionar ao renderizar

BrowserRouter x HashRouter

BrowserRouter

- URLs limpas: `/alunos/42`
- Exige *rewrite* no servidor (`vercel.json`, nginx)
- Padrão dos projetos

vs

HashRouter

- URLs com `#`: `/#/alunos/42`
- Funciona em qualquer hospedagem estática sem configuração
- Útil no GitHub Pages

Organização: barrel files e caminhos

Barrel file

```
// src/components/index.js
export { default as Menu } from "./Menu";
export { default as Botao } from "./Botao";
export { default as Message } from "./Message";

// em uma página: um import só
import { Menu, Botao, Message } from "../components";
```

Aprofundando: rotas aninhadas e layouts

Dica

Ao voltar de uma rota protegida, use o `state` guardado pelo `ProtectedRoute` para levar o usuário de volta à página que ele tentou abrir: `navigate(location.state?.de ?? "/")` depois do login.

07

Quiz de revisão

Quiz · questão 1 de 5

Pergunta

Por que não usar `` para links internos?

A Não funciona

B Recarrega a página inteira e perde o estado

C É lento no Vite

D Quebra o CSS

Quiz · questão 1 de 5

Resposta

Por que não usar `` para links internos?

A Não funciona

✓ Recarrega a página inteira e perde o estado

C É lento no Vite

D Quebra o CSS

Por quê: Use Link ou NavLink.

Quiz · questão 2 de 5

Pergunta

Qual hook lê o `:id` de `/alunos/:id`?

A useLocation

B useParams

C useNavigate

D useState

Resposta

Qual hook lê o `:id` de `/alunos/:id`?

A useLocation

✓ useParams

C useNavigate

D useState

Por quê: useNavigate navega por código.

Pergunta

O que `` faz?

A Rota inicial

B Captura endereços não cadastrados (404)

C Rota protegida

D Redireciona para o login

Quiz · questão 3 de 5

Resposta

O que `` faz?

A Rota inicial

✓ Captura endereços não cadastrados (404)

C Rota protegida

D Redireciona para o login

Por quê: Coloque por último.

Pergunta

F5 em `/alunos` dá 404 no Vercel. Causa:

A. Falta o React Router

B. Falta o rewrite para index.html (vercel.json)

C. Erro de CORS

D. Falta o key

Resposta

F5 em `/alunos` dá 404 no Vercel. Causa:

A. Falta o React Router

✓ Falta o rewrite para index.html (vercel.json)

C. Erro de CORS

D. Falta o key

Por quê: No dev o Vite já faz esse fallback.

Pergunta

A rota protegida no front garante a segurança dos dados?

A Sim

B Não: ela só esconde telas; o back precisa exigir o token

C Só com HTTPS

D Só no Vercel

Resposta

A rota protegida no front garante a segurança dos dados?

A Sim

✓ Não: ela só esconde telas; o back precisa exigir o token

C Só com HTTPS

D Só no Vercel

Por quê: Qualquer um pode ler o JavaScript do site.

Fechamento

Resumo da aula

- ✓ `react-router-dom`: `BrowserRouter` → `Routes` → `Route path element`; `*` para 404.
- ✓ Links internos com `Link`/`NavLink`; parâmetros com `useParams`; navegação com `useNavigate`.
- ✓ `ProtectedRoute` redireciona com ``, mas a segurança real é no back-end.
- ✓ No deploy, toda rota precisa cair no `index.html` (`vercel.json` / nginx).
- ✓ Pastas: `pages`, `components`, `services`, `hooks`, `config`, `utils`.

Exercícios

1. Crie um site com Início, Sobre, Contato e uma página 404.

2. Liste alunos em `/alunos` e mostre os detalhes em `/alunos/:id`.

3. Proteja `/alunos` com um login falso que grava o usuário no `localStorage`.

4. Reorganize um projeto seu seguindo a tabela de pastas e escreva um README com a estrutura.

Para ir além

- reactrouter.com/start
- Projetos `SecureLoginPUC_React`, `Pokedex`, `PucMaps`

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 08?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web